

Arquivo 03 - VIEW:

- Criação da VIEW: vw_historico_academico
- Descrição: Pesquisa os dados de todos os alunos para facilitar consultas.
- Informações da pesquisa: nome do aluno, curso, professor e da disciplina, período da disciplina, nota, frequência e situação.

```
7 • CREATE OR REPLACE VIEW vw_historico_academico AS
8     SELECT
9         a.nome AS aluno,
10        c.nome AS curso,
11        d.nome AS disciplina,
12        CONCAT(t.semestre, '/', t.ano) AS periodo,
13        p.nome AS professor,
14        m.nota AS nota,
15        m.frequencia AS frequencia,
16        m.situacao AS situacao
17    FROM matriculas m
18     JOIN alunos a ON m.id_aluno = a.id_aluno
19     JOIN cursos c ON a.id_curso = c.id_curso
20     JOIN turmas t ON m.id_turma = t.id_turma
21     JOIN disciplinas d ON t.id_disc = d.id_disc
22     JOIN professores p ON t.id_prof = p.id_prof
23    ORDER BY aluno;
24
25
26 • SELECT * FROM vw_historico_academico;
```

Result Grid		Filter Rows:	Export:	Wrap Cell Content:			
aluno	curso	disciplina	periodo	professor	nota	frequencia	situacao
▶ Arthur	Engenharia de Computação	Banco de Dados 2	1/2026	Salvador	9,0	100,00	Aprovado
Arthur	Engenharia de Computação	Estrutura de Dados	1/2027	Vera	NULL	NULL	Em Curso
Arthur	Engenharia de Computação	Processamento de Linguagem Natural	2/2026	Ednardo	NULL	NULL	Em Curso
Carlos	Engenharia de Computação	Estrutura de Dados	1/2027	Vera	NULL	NULL	Em Curso
Emanuel	Ciência da Computação	Sistemas embarcados / Microprocessadores	2/2026	Hudson	8,7	20,00	Reprovado por Falta
Felipe	Ciência da Computação	Banco de Dados 2	1/2026	Salvador	7,8	90,00	Aprovado
Gabriel	Engenharia de Computação	Banco de Dados 2	1/2026	Salvador	NULL	NULL	Em Curso
Gabriel	Engenharia de Computação	Circuitos Digitais	1/2026	Hudson	4,3	100,00	Reprovado por Nota
Gabriel	Engenharia de Computação	Sistemas embarcados / Microprocessadores	2/2026	Hudson	NULL	NULL	Em Curso

o_academico 1 x

#	Time	Action	Message
✓ 18	09:35:29	INSERT INTO matriculas(id_aluno, id_turma, nota, frequencia, situacao) VALUES (1, 1, 9, 100, 'Aprovado'). (2, 1, 7.8, 90, 'Aprovado'). (3, 2, 8.1, ...	7 row(s) affected Records: 7 Duplicates: 0 Warnings: 0
✓ 19	09:35:29	INSERT INTO matriculas(id_aluno, id_turma) VALUES (8, 4), (9, 5), (10, 5), (1, 4), (1, 5), (4, 3), (4, 5), (4, 1), (5, 5), (6, 5)	10 row(s) affected Records: 10 Duplicates: 0 Warnings: 0
✓ 20	09:35:33	CREATE OR REPLACE VIEW vw_historico_academico AS SELECT a.nome AS aluno, c.nome AS curso, d.nome AS disciplina, CONCAT(t.s...	0 row(s) affected
✓ 21	09:35:33	SELECT * FROM vw_historico_academico LIMIT 0, 1000	17 row(s) returned

Arquivo 04 - TRIGGER:

- Criação da Trigger: trg_calcula_situacao
- Momento: Antes do UPDATE.
- Objetivo: Atualizar automaticamente a coluna 'situacao' da tabela 'matriculas' sempre que a nota E frequência forem alteradas.

```
9 • CREATE TRIGGER trg_calcula_situacao
10 BEFORE UPDATE ON matriculas
11 FOR EACH ROW
12 BEGIN
13
14 IF NEW.nota IS NOT NULL AND NEW.frequencia IS NOT NULL THEN
15
16 IF NEW.frequencia < 75 THEN
17 SET NEW.situacao = 'Reprovado por falta';
18 ELSEIF NEW.frequencia >= 75 AND NEW.nota < 6 THEN
19 SET NEW.situacao = 'Reprovado por Nota';
20 ELSE
21 SET NEW.situacao = 'Aprovado';
22 END IF;
23
24 ELSE
25
26 SIGNAL SQLSTATE '45000'
27 SET MESSAGE_TEXT = 'Algum dos valores é nulo.';
28
29 END IF;
30
31 END;
```

	id_mat	id_aluno	id_turma	dt_inscricao	nota	frequencia	situacao
	5	5	3	2026-06-15	2.9	100.00	Reprovado por Nota
	6	6	3	2026-06-15	10.0	30.00	Reprovado por Falta
	7	7	3	2026-06-15	8.7	20.00	Reprovado por Falta
	8	8	4	2026-06-15	7.0	70.00	Reprovado por Falta
	9	9	5	2026-06-15	5.8	80.00	Reprovado por Nota
	10	10	5	2026-06-15	8.2	100.00	Aprovado
	11	1	4	2026-06-15	NULL	NULL	Em Curso

matriculas 3 x

Output			
Action Output			
#	Time	Action	Message
✓ 25	09:40:30	UPDATE matriculas SET frequencia = 80, nota = 5.8 WHERE id_mat = 9	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0
✓ 26	09:40:30	SELECT * FROM matriculas LIMIT 0, 1000	17 row(s) returned
✓ 27	09:40:33	UPDATE matriculas SET frequencia = 100, nota = 8.2 WHERE id_mat = 10	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0
✓ 28	09:40:45	SELECT * FROM matriculas LIMIT 0, 1000	17 row(s) returned

Arquivo 05 - QUERIES:

- Queries de 1 a 10 que fazem consultas específicas no Banco de Dados, solicitadas no projeto.

QUERY 1: Todos os alunos com nome do curso e total de disciplinas cursadas.

```
6
7  -- QUERY 1:
8  -- Todos os alunos com nome do curso e total de disciplinas cursadas.
9  • SELECT
10     a.nome AS aluno,
11     c.nome AS curso,
12     COUNT(id_disc) AS disciplinas
13 FROM
14     alunos a
15 JOIN
16     cursos c ON c.id_curso = a.id_curso
17 JOIN
18     matriculas m ON m.id_aluno = a.id_aluno
19 JOIN
20     turmas t ON t.id_turma = m.id_turma
21 GROUP BY a.id_aluno;
```

aluno	curso	disciplinas
Arthur	Engenharia de Computação	3
Gabriel	Engenharia de Computação	4
Carlos	Engenharia de Computação	1
Manuela	Engenharia de Computação	1
Felipe	Ciência da Computação	1
Gustavo	Ciência da Computação	2
Emanuel	Ciência da Computação	1
João	Inteligência Artificial e Machine Learning	1
Gil	Inteligência Artificial e Machine Learning	2
Luiz	Inteligência Artificial e Machine Learning	1

Result 1 ×

Output

Action Output

#	Time	Action	Message
✓ 26	09:40:30	SELECT * FROM matriculas LIMIT 0, 1000	17 row(s) returned
✓ 27	09:40:33	UPDATE matriculas SET frequencia = 100, nota = 8.2 WHERE id_mat = 10	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0
✓ 28	09:40:45	SELECT * FROM matriculas LIMIT 0, 1000	17 row(s) returned
✓ 29	09:50:06	SELECT a.nome AS aluno, c.nome AS curso, COUNT(id_disc) AS disciplinas FROM alunos a JOIN cursos c ON c.id_curso = a.id_curso ...	10 row(s) returned

QUERRY 2: Professores com mais de 2 turmas ativas no ano corrente.

```
24
25 -- QUERRY 2:
26 -- Professores com mais de 2 turmas ativas no ano corrente.
27 • SELECT
28     p.nome AS professor,
29     COUNT(t.id_turma) AS turmas
30 FROM
31     professores p
32 JOIN
33     turmas t ON p.id_prof = t.id_prof
34 WHERE ano = YEAR(NOW())
35 GROUP BY p.id_prof
36 HAVING COUNT(id_turma) > 2;
37
```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
professor	turmas		
Hudson	3		

Result 2 x

Output			
Action Output			
#	Time	Action	Message
✓ 27	09:40:33	UPDATE matriculas SET frequencia = 100, nota = 8.2 WHERE id_mat = 10	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0
✓ 28	09:40:45	SELECT * FROM matriculas LIMIT 0, 1000	17 row(s) returned
✓ 29	09:50:06	SELECT a.nome AS aluno, c.nome AS curso, COUNT(id_disc) AS disciplinas FROM alunos a JOIN cursos c ON c.id_curso = a.id_curso ...	10 row(s) returned
✓ 30	09:52:09	SELECT p.nome AS professor, COUNT(t.id_turma) AS turmas FROM professores p JOIN turmas t ON p.id_prof = t.id_prof WHERE ano = YEAR(N...	1 row(s) returned

QUERY 3: Taxa de aprovação por disciplina (%).

```
39
40 -- QUERY 3:
41 -- Taxa de aprovação por disciplina (%).
42 • SELECT
43     d.nome AS disciplina,
44     ROUND(SUM(CASE WHEN m.situacao = 'Aprovado' THEN 1 ELSE 0 END) * 100 / COUNT(*), 2) AS taxa_aprovacao
45 FROM
46     matriculas m
47 JOIN
48     turmas t ON t.id_turma = m.id_turma
49 RIGHT JOIN
50     disciplinas d ON d.id_disc = t.id_disc
51 GROUP BY d.id_disc;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	disciplina	taxa_aprovacao
▶	Banco de Dados 1	0.00
	Banco de Dados 2	66.67
	Processamento de Linguagem Natural	16.67
	Sistemas embarcados / Microprocessadores	0.00
	Circuitos Digitais	50.00
	Bootcamp 1	0.00
	Engenharia de Requisitos	0.00
	Estrutura de Dados	0.00
	Programação Orientada a Objeto	0.00
	Lógica de Programação	0.00
	Bootcamp 2	0.00

Result 3 x

Output

Action Output

#	Time	Action	Message
✓ 28	09:40:45	SELECT * FROM matriculas LIMIT 0, 1000	17 row(s) returned
✓ 29	09:50:06	SELECT a.nome AS aluno, c.nome AS curso, COUNT(id_disc) AS disciplinas FROM alunos a JOIN cursos c ON c.id_curso = a.id_curso ...	10 row(s) returned
✓ 30	09:52:09	SELECT p.nome AS professor, COUNT(t.id_turma) AS turmas FROM professores p JOIN turmas t ON p.id_prof = t.id_prof WHERE ano = YEAR(N...	1 row(s) returned
✓ 31	09:53:27	SELECT d.nome AS disciplina, ROUND(SUM(CASE WHEN m.situacao = 'Aprovado' THEN 1 ELSE 0 END) * 100 / COUNT(*), 2) AS taxa_aprova...	11 row(s) returned

QUERY 4: Alunos com nota acima da média da sua disciplina. Usando Subquery.

```
87  -- Query 4:
88  -- Alunos com nota acima da média da sua disciplina
89  -- Usando Subquery.
90  • SELECT
91      d.nome AS disciplina,
92      a.nome AS aluno,
93      m.nota
94  FROM
95      matriculas m
96  JOIN
97      alunos a ON a.id_aluno = m.id_aluno
98  JOIN
99      turmas t ON t.id_turma = m.id_turma
100  JOIN
101      disciplinas d ON d.id_disc = t.id_disc
102  WHERE m.nota > (
103      SELECT AVG(m2.nota)
104      FROM matriculas m2
105      JOIN turmas t2 ON t2.id_turma = m2.id_turma
106      WHERE t2.id_disc = t.id_disc
107  );
108
109
110
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	disciplina	aluno	nota
▶	Banco de Dados 2	Arthur	9.0
	Circuitos Digitais	João	8.1
	Sistemas embarcados / Microprocessadores	Gustavo	10.0
	Sistemas embarcados / Microprocessadores	Emanuel	8.7
	Processamento de Linguagem Natural	Manuela	8.2

Result 4 x

Output

Action Output

#	Time	Action	Message
✓ 29	09:50:06	SELECT a.nome AS aluno, c.nome AS curso, COUNT(id_disc) AS disciplinas FROM alunos a JOIN cursos c ON c.id_curso = a.id_curso ...	10 row(s) returned
✓ 30	09:52:09	SELECT p.nome AS professor, COUNT(t.id_turma) AS turmas FROM professores p JOIN turmas t ON p.id_prof = t.id_prof WHERE ano = YEAR(N...	1 row(s) returned
✓ 31	09:53:27	SELECT d.nome AS disciplina, ROUND(SUM(CASE WHEN m.situacao = 'Aprovado' THEN 1 ELSE 0 END) * 100 / COUNT(*), 2) AS taxa_aprova...	11 row(s) returned
✓ 32	09:55:35	SELECT d.nome AS disciplina, a.nome AS aluno, m.nota FROM matriculas m JOIN alunos a ON a.id_aluno = m.id_aluno JOIN turmas...	5 row(s) returned

QUERY 5: Disciplinas sem nenhuma matrícula ativa.

```
111 -- QUERY 5:
112 -- Disciplinas sem nenhuma matrícula ativa.
113 • SELECT
114     d.nome AS disciplina
115 FROM
116     disciplinas d
117     LEFT JOIN
118     turmas t ON t.id_disc = d.id_disc
119     LEFT JOIN
120     matriculas m ON m.id_turma = t.id_turma
121 WHERE m.id_mat IS NULL;
122
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

disciplina
▶ Banco de Dados 1
Bootcamp 1
Engenharia de Requisitos
Programação Orientada a Objeto
Lógica de Programação
Bootcamp 2

Result 5 x

Output

Action Output

#	Time	Action	Message
✓ 30	09:52:09	SELECT p.nome AS professor, COUNT(t.id_turma) AS turmas FROM professores p JOIN turmas t ON p.id_prof = t.id_prof WHERE ano = YEAR(N...	1 row(s) returned
✓ 31	09:53:27	SELECT d.nome AS disciplina, ROUND(SUM(CASE WHEN m.situacao = 'Aprovado' THEN 1 ELSE 0 END) * 100 / COUNT(*), 2) AS taxa_aprova...	11 row(s) returned
✓ 32	09:55:35	SELECT d.nome AS disciplina, a.nome AS aluno, m.nota FROM matriculas m JOIN alunos a ON a.id_aluno = m.id_aluno JOIN turmas...	5 row(s) returned
✓ 33	09:57:33	SELECT d.nome AS disciplina FROM disciplinas d LEFT JOIN turmas t ON t.id_disc = d.id_disc LEFT JOIN matriculas m ON m.id_turma...	6 row(s) returned

QUERRY 6: Ranking de alunos por média ponderada (créditos).

```

126 -- QUERRY 6:
127 -- Ranking de alunos por média ponderada (créditos).
128 • SELECT
129     a.nome AS aluno,
130     ROUND(SUM(m.nota * d.creditos) / SUM(d.creditos), 2) AS media_ponderada
131 FROM
132     alunos a
133 JOIN
134     matriculas m ON m.id_aluno = a.id_aluno
135 JOIN
136     turmas t ON t.id_turma = m.id_turma
137 JOIN
138     disciplinas d ON d.id_disc = t.id_disc
139 GROUP BY a.id_aluno
140 ORDER BY media_ponderada DESC;

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

	aluno	media_ponderada
▶	Emanuel	8.70
	Manuela	8.20
	João	8.10
	Felipe	7.80
	Carlos	7.00
	Gustavo	6.00
	Luiz	5.80
	Arthur	2.57
	Gil	1.74
	Gabriel	1.29

Result 6 ×

Output

Action Output

#	Time	Action	Message
✓ 31	09:53:27	SELECT d.nome AS disciplina, ROUND(SUM(CASE WHEN m.situacao = 'Aprovado' THEN 1 ELSE 0 END) * 100 / COUNT(*), 2) AS taxa_aprova...	11 row(s) returned
✓ 32	09:55:35	SELECT d.nome AS disciplina, a.nome AS aluno, m.nota FROM matriculas m JOIN alunos a ON a.id_aluno = m.id_aluno JOIN turmas...	5 row(s) returned
✓ 33	09:57:33	SELECT d.nome AS disciplina FROM disciplinas d LEFT JOIN turmas t ON t.id_disc = d.id_disc LEFT JOIN matriculas m ON m.id_turma...	6 row(s) returned
✓ 34	09:59:14	SELECT a.nome AS aluno, ROUND(SUM(m.nota * d.creditos) / SUM(d.creditos), 2) AS media_ponderada FROM alunos a JOIN matriculas...	10 row(s) returned

QUERY 7: Disciplinas e seus pré-requisitos via Self JOIN.

```
144 -- QUERY 7:
145 -- Disciplinas e seus pré-requisitos via Self JOIN.
146 • SELECT
147     d.nome AS disciplina,
148     pr.nome AS pre_requisito
149 FROM
150     disciplinas d
151 JOIN
152     pre_requisitos p ON p.id_disc = d.id_disc
153 JOIN
154     disciplinas pr ON pr.id_disc = p.id_pre_req;
```

Result Grid   Filter Rows: Export:  Wrap Cell Content: 

	disciplina	pre_requisito
▶	Banco de Dados 2	Banco de Dados 1
	Bootcamp 2	Bootcamp 1
	Programação Orientada a Objeto	Lógica de Programação

Result 7 x

Output

 Action Output ▼

#	Time	Action	Message
✓ 32	09:55:35	SELECT d.nome AS disciplina, a.nome AS aluno, m.nota FROM matriculas m JOIN alunos a ON a.id_aluno = m.id_aluno JOIN turmas...	5 row(s) returned
✓ 33	09:57:33	SELECT d.nome AS disciplina FROM disciplinas d LEFT JOIN turmas t ON t.id_disc = d.id_disc LEFT JOIN matriculas m ON m.id_tum...	6 row(s) returned
✓ 34	09:59:14	SELECT a.nome AS aluno, ROUND(SUM(m.nota * d.creditos) / SUM(d.creditos), 2) AS media_ponderada FROM alunos a JOIN matriculas...	10 row(s) returned
✓ 35	10:00:31	SELECT d.nome AS disciplina, pr.nome AS pre_requisito FROM disciplinas d JOIN pre_requisitos p ON p.id_disc = d.id_disc JOIN dis...	3 row(s) returned

QUERY 8: Cursos com média de nota dos alunos abaixo de 7,0.

```
158 -- QUERY 8:
159 -- Cursos com média de nota dos alunos abaixo de 7,0.
160 • SELECT
161     c.nome AS curso,
162     AVG(m.nota) AS media
163 FROM
164     disciplinas d
165 JOIN
166     turmas t ON t.id_disc = d.id_disc
167 JOIN
168     matriculas m ON m.id_turma = t.id_turma
169 JOIN
170     alunos a ON a.id_aluno = m.id_aluno
171 JOIN
172     cursos c ON c.id_curso = a.id_curso
173 GROUP BY c.id_curso
174 HAVING AVG(m.nota) < 7;
175
```

Result Grid | | Filter Rows: | Export: | Wrap Cell Content:

	curso	media
▶	Inteligência Artificial e Machine Learning	5.60000

Result 8 ×

Output

Action Output

	#	Time	Action	Message
✓	33	09:57:33	SELECT d.nome AS disciplina FROM disciplinas d LEFT JOIN turmas t ON t.id_disc = d.id_disc LEFT JOIN matriculas m ON m.id_turma...	6 row(s) returned
✓	34	09:59:14	SELECT a.nome AS aluno, ROUND(SUM(m.nota * d.creditos) / SUM(d.creditos), 2) AS media_ponderada FROM alunos a JOIN matriculas...	10 row(s) returned
✓	35	10:00:31	SELECT d.nome AS disciplina, pr.nome AS pre_requisito FROM disciplinas d JOIN pre_requisitos p ON p.id_disc = d.id_disc JOIN dis...	3 row(s) returned
✓	36	10:01:41	SELECT c.nome AS curso, AVG(m.nota) AS media FROM disciplinas d JOIN turmas t ON t.id_disc = d.id_disc JOIN matriculas m ON ...	1 row(s) returned

QUERY 9: Histórico completo de um aluno específico via view

```

178 -- QUERY 9:
179 -- Histórico completo de um aluno específico via view
180 • SELECT * FROM vw_historico_academico
181 WHERE aluno = 'Arthur';
182

```

Result Grid		Filter Rows:		Export:	Wrap Cell Content:			
	aluno	curso	disciplina	periodo	professor	nota	frequencia	situacao
▶	Arthur	Engenharia de Computação	Banco de Dados 2	1/2026	Salvador	9.0	100.00	Aprovado
	Arthur	Engenharia de Computação	Estrutura de Dados	1/2027	Vera	NULL	NULL	Em Curso
	Arthur	Engenharia de Computação	Processamento de Linguagem Natural	2/2026	Ednardo	NULL	NULL	Em Curso

vw_historico_academico 9 x

Output

Action Output

#	Time	Action	Message
✓ 34	09:59:14	SELECT a.nome AS aluno, ROUND(SUM(m.nota * d.creditos) / SUM(d.creditos), 2) AS media_ponderada FROM alunos a JOIN matriculas...	10 row(s) returned
✓ 35	10:00:31	SELECT d.nome AS disciplina, pr.nome AS pre_requisito FROM disciplinas d JOIN pre_requisitos p ON p.id_disc = d.id_disc JOIN dis...	3 row(s) returned
✓ 36	10:01:41	SELECT c.nome AS curso, AVG(m.nota) AS media FROM disciplinas d JOIN turmas t ON t.id_disc = d.id_disc JOIN matriculas m ON ...	1 row(s) returned
✓ 37	10:02:52	SELECT * FROM vw_historico_academico WHERE aluno = 'Arthur' LIMIT 0, 1000	3 row(s) returned

QUERRY 10: Professores que nunca reprovaram nenhum aluno

```

186 -- QUERRY 10:
187 -- Professores que nunca reprovaram nenhum aluno
188 • SELECT
189     p.nome AS professor
190 FROM
191     professores p
192 WHERE NOT EXISTS (
193     SELECT 1
194     FROM turmas t
195     JOIN matriculas m ON t.id_turma = m.id_turma
196     WHERE t.id_prof = p.id_prof
197           AND m.situacao IN('Reprovado por Nota', 'Reprovado por Falta')
198 );

```

Result Grid  Filter Rows: Export:  Wrap Cell Content: 

professor
▶ Salvador
Molina

professores 10 x

Output

Action Output

#	Time	Action	Message
✓ 35	10:00:31	SELECT d.nome AS disciplina, pr.nome AS pre_requisito FROM disciplinas d JOIN pre_requisitos p ON p.id_disc = d.id_disc JOIN dis...	3 row(s) returned
✓ 36	10:01:41	SELECT c.nome AS curso, AVG(m.nota) AS media FROM disciplinas d JOIN turmas t ON t.id_disc = d.id_disc JOIN matriculas m ON ...	1 row(s) returned
✓ 37	10:02:52	SELECT * FROM vw_historico_academico WHERE aluno = 'Arthur' LIMIT 0, 1000	3 row(s) returned
✓ 38	10:03:48	SELECT p.nome AS professor FROM professores p WHERE NOT EXISTS (SELECT 1 FROM turmas t JOIN matriculas m ON t.id_t...	2 row(s) returned

Arquivo 06 - TRANSACOES: Possui 2 cenários de transações usando BEGIN, COMMIT e ROLLBACK.

CENÁRIO 1: Matrícula com rollback por capacidade excedida.

- Criação da Procedure: inserir_matricula_turma

- Objetivo: Facilitar o cenário 1 na realização da matrícula de um aluno em uma turma.

Parâmetros:

in_id_aluno (INT): ID do aluno que será matriculado;

in_id_turma (INT): ID da turma de destino.

```
18
19 CREATE PROCEDURE inserir_matricula_turma(
20     IN in_id_aluno INT,
21     IN in_id_turma INT
22 )
23 BEGIN
24     DECLARE vagas_restantes INT DEFAULT 0;
25     -- variável para cálculo de vagas na turma.
26
27     -- Inicia o controle de transação garantindo atomicidade.
28     START TRANSACTION;
29
30     -- Cálculo das vagas restantes na turma.
31     SELECT (t.capacidade - COUNT(m.id_mat)) INTO vagas_restantes
32     FROM turmas t
33     LEFT JOIN matriculas m ON m.id_turma = t.id_turma
34     WHERE t.id_turma = in_id_turma
35     GROUP BY t.id_turma;
36
37     -- Verificação da quantidade de vagas restantes.
38     IF vagas_restantes > 0 THEN
39         INSERT INTO matriculas(id_aluno, id_turma)
40         VALUES (in_id_aluno, in_id_turma);
41         COMMIT;
42         -- COMMIT do aluno na turma desejada.
43         SELECT 'Sucesso na operação' AS status;
44     ELSE
45         ROLLBACK;
46         -- ROLLBACK não realizando matrícula do aluno na turma desejada.
47         SELECT 'ERRO: Turma lotada. Matrícula não realizada.' AS status;
48     END IF;
49 END;
--
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

status

► Sucesso na operação

Result 1 x

Output

Action Output

#	Time	Action	Message
✓ 37	10:02:52	SELECT * FROM vw_historico_academico WHERE aluno = 'Arthur' LIMIT 0, 1000	3 row(s) returned
✓ 38	10:03:48	SELECT p.nome AS professor FROM professores p WHERE NOT EXISTS (SELECT 1 FROM turmas t JOIN matriculas m ON t.id_turma = ...	2 row(s) returned
✓ 39	10:05:26	CREATE PROCEDURE inserir_matricula_turma(IN in_id_aluno INT, IN in_id_turma INT) BEGIN DECLARE vagas_restantes INT DEFAULT 0; -- v...	0 row(s) affected
✓ 40	10:05:26	-- Primeira transação na turma com capacidade para 1 aluno. CALL inserir_matricula_turma(6, 8)	1 row(s) returned

Erro Cenário 1 turma lotada:

```
56 -- Segunda transação com a turma lotada devido à primeira transação.
57 • CALL inserir_matricula_turma(1, 8);
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: [IA](#)

status
ERRO: Turma lotada. Matrícula não realizada.

Result 2 x

Output

Action Output

#	Time	Action	Message
✓ 38	10:03:48	SELECT p.nome AS professor FROM professores p WHERE NOT EXISTS (SELECT 1 FROM turmas t JOIN matriculas m ON t.id_turma = ...	2 row(s) returned
✓ 39	10:05:26	CREATE PROCEDURE inserir_matricula_turma(IN in_id_aluno INT, IN in_id_turma INT) BEGIN DECLARE vagas_restantes INT DEFAULT 0; -- v...	0 row(s) affected
✓ 40	10:05:26	-- Primeira transação na turma com capacidade para 1 aluno. CALL inserir_matricula_turma(6, 8)	1 row(s) returned
✓ 41	10:10:10	CALL inserir_matricula_turma(1, 8)	1 row(s) returned

CENÁRIO 2: Lançamento de notas em lote.

```
74 -- Primeira transaction para lançamentos de notas com COMMIT.
75 • START TRANSACTION;
76
77 • UPDATE matriculas
78 SET nota = 8.0, frequencia = 70
79 WHERE id_mat = 11;
80
81 • UPDATE matriculas
82 SET nota = 5, frequencia = 80
83 WHERE id_mat = 12;
84
85 • UPDATE matriculas
86 SET nota = 9, frequencia = 90
87 WHERE id_mat = 13;
88
89 • SELECT id_mat, situacao
90 FROM matriculas
91 WHERE id_mat IN(11, 12, 13);
92
93 • COMMIT;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: [IA](#)

id_mat	situacao
11	Reprovado por Falta
12	Reprovado por Nota
13	Aprovado
NULL	NULL

matriculas 3 x

Output

Action Output

#	Time	Action	Message
✓ 44	10:12:02	UPDATE matriculas SET nota = 5, frequencia = 80 WHERE id_mat = 12	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0
✓ 45	10:12:02	UPDATE matriculas SET nota = 9, frequencia = 90 WHERE id_mat = 13	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0
✓ 46	10:12:02	SELECT id_mat, situacao FROM matriculas WHERE id_mat IN(11, 12, 13) LIMIT 0, 1000	3 row(s) returned
✓ 47	10:12:02	COMMIT	0 row(s) affected

SELECT antes do ROLLBACK Cenário 2:

```
97 -- Segunda transaction com ROLLBACK e verificação antes e depois do ROLLBACK
98 • START TRANSACTION;
99
100 • UPDATE matriculas
101   SET nota = 0, frequencia = 0
102   WHERE id_mat IN (1, 2, 3);
103
104 -- Verificação antes do ROLLBACK
105 • SELECT id_mat, nota, frequencia, situacao FROM matriculas WHERE id_mat IN (1, 2, 3);
106
107 • ROLLBACK;
108
109 -- Verificação depois do ROLLBACK
110 • SELECT id_mat, nota, frequencia, situacao FROM matriculas WHERE id_mat IN (1, 2, 3);
```

Result Grid				
Filter Rows:				
Edit: Export/Import: Wrap Cell Content:				
	id_mat	nota	frequencia	situacao
▶	1	0.0	0.00	Reprovado por Falta
	2	0.0	0.00	Reprovado por Falta
	3	0.0	0.00	Reprovado por Falta
•	NULL	NULL	NULL	NULL

matriculas 4 x

Output			
Action Output			
#	Time	Action	Message
✓ 48	10:13:23	START TRANSACTION	0 row(s) affected
✓ 49	10:13:23	UPDATE matriculas SET nota = 0, frequencia = 0 WHERE id_mat IN (1, 2, 3)	3 row(s) affected Rows matched: 3 Changed: 3 Warnings: 0
✓ 50	10:13:23	SELECT id_mat, nota, frequencia, situacao FROM matriculas WHERE id_mat IN (1, 2, 3) LIMIT 0, 1000	3 row(s) returned
✓ 51	10:13:23	ROLLBACK	0 row(s) affected

SELECT depois do ROLLBACK Cenário 2:

```
109 -- Verificação depois do ROLLBACK
110 • SELECT id_mat, nota, frequencia, situacao FROM matriculas WHERE id_mat IN (1, 2, 3);
```

Result Grid				
Filter Rows:				
Edit: Export/Import: Wrap Cell Content:				
	id_mat	nota	frequencia	situacao
▶	1	9.0	100.00	Aprovado
	2	7.8	90.00	Aprovado
	3	8.1	80.00	Aprovado
•	NULL	NULL	NULL	NULL

matriculas 5 x

Output			
Action Output			
#	Time	Action	Message
✓ 49	10:13:23	UPDATE matriculas SET nota = 0, frequencia = 0 WHERE id_mat IN (1, 2, 3)	3 row(s) affected Rows matched: 3 Changed: 3 Warnings: 0
✓ 50	10:13:23	SELECT id_mat, nota, frequencia, situacao FROM matriculas WHERE id_mat IN (1, 2, 3) LIMIT 0, 1000	3 row(s) returned
✓ 51	10:13:23	ROLLBACK	0 row(s) affected
✓ 52	10:14:59	SELECT id_mat, nota, frequencia, situacao FROM matriculas WHERE id_mat IN (1, 2, 3) LIMIT 0, 1000	3 row(s) returned